

PART VI — THE SELF-LEARNING LAYER

Chapter 30

Learning From Failure, Part 1: Failed Query Variants

“The cheapest lesson a system can learn is which questions it already tried and already lost.”

Chapter 30 — Learning From Failure, Part 1: Failed Query Variants

This chapter builds `failed_variants.json` — a blocklist keyed by normalized query text — and traces the full loop: recording a failure during a run, persisting it at the end of the session, loading it back at the start of the next one, and enforcing it as a hard filter rather than a hopeful prompt instruction. Along the way, two real bugs from this project's own LangGraph port — a missing write-back and a broken file path — show exactly what happens when one half of that loop is implemented and the other is quietly forgotten.

30.1 The problem

Chapter 23.4 already quantified the cost of a single retrieval call — roughly 1,250 tokens for a top-k=5 result. A query phrasing that reliably returns zero chunks costs exactly that much every time it is tried, for exactly zero benefit, and nothing about the agent's own reasoning prevents it from reaching for a phrasing it — or a **previous** invocation of the same agent — already exhausted. `_PROCESS_INSTRUCTIONS`'s within-session dedup (Chapter 27.4's `all_tried_queries` set) stops a single run from repeating itself, but a fresh process start has no memory of what the last run already learned.

The practical failure mode is a query like "What is the recommended dosage of risperidone for a 7-year-old child with autism and epilepsy?" — genuinely outside this project's ASD-and-Adjustable-Speed-Drive corpus, and genuinely certain to retrieve nothing no matter how many times or how many different ways it gets asked. `run_batch.py`'s Batch 3 ("zero_chunk_situations") exists specifically to exercise this case. Without cross-session memory, every user who asks a variant of that same out-of-corpus question pays the full multi-reformulation retrieval cost the very first user already paid.

The cost compounds in a way that is easy to underestimate from a single query's perspective. A knowledge base with a stable, unchanging scope will keep encountering the same handful of out-of-domain question categories indefinitely — medication dosing, brand-specific product comparisons, and regulatory codes were this project's own recurring zero-chunk categories, per Batch 3's design. Every user who independently stumbles into one of those categories re-pays the discovery cost from scratch unless something durable remembers that the knowledge base has already been asked, and has already failed to answer, a question shaped like this one.

30.2 The blocklist file

`failed_variants.json` is keyed by **normalized** query text — lowercased and stripped of surrounding whitespace, the identical normalization `FeedbackStore.normalize_query()` applies to thumbdown lookups (Chapter 31.10) — mapping each normalized query to the list of prior failed answers or a simple count, depending on how much detail a given deployment wants to retain. Keying by normalized text rather than a semantic embedding is a deliberate simplicity choice: exact-match lookup on a string is $O(1)$ with a plain dict, requires no model call, and catches the single most common repeat-failure case — the agent or a **different** user asking the literally identical question — without the false-positive risk fuzzy matching would introduce.

```
{
  "what is the recommended dosage of risperidone for a 7-year-old child
  with autism and epilepsy?": {
    "count": 3,
    "last_seen": "2026-06-18T14:22:03Z"
  },
  "which specific brand of variable frequency drive is best for a 75 kw
  pump application?": {
    "count": 1,
    "last_seen": "2026-06-11T09:03:47Z"
  }
}
```

Section 30.10 (carried forward properly in Chapter 31.10) revisits the exact-vs-fuzzy trade-off this key design commits to: a query that fails as "dosage of risperidone for autism" and later gets asked as "risperidone dose autism child" will not match this blocklist at all, since the two strings normalize differently. This is a known, accepted limitation of exact keying, not an oversight — Chapter 31.10 explains why a fuzzy-matching upgrade is a real option but not a free one.

Storing a count and a last-seen timestamp per entry, rather than a bare boolean "has failed," is a small design choice with a real payoff: a query that has failed once might be an unlucky phrasing worth one more attempt with a different embedding model or a corpus update; a query that has failed a dozen times across a month is a far stronger signal that no reformulation is likely to help. Neither this chapter's minimal implementation nor the original project's shipped version acts on that distinction automatically, but the data is there to support exactly that kind of tiered policy — block immediately past some count threshold, but tolerate a first or second occurrence — without needing to change the record schema later.

30.3 Recording every failing reformulation

A design that only records the query a user explicitly thumbs-down misses the majority of the actual signal. Within a single multi-iteration run, `\_PROCESS\_INSTRUCTIONS` has the agent try 2-3 semantically different phrasings before compression — if two of those three retrieve nothing and the third succeeds, only the user ever sees the successful answer, but all three phrasings are real evidence about what does and does not work for this topic. Recording only the user-flagged failure would discard the two silent failures that happened inside a session that technically "succeeded."

This project's own LangGraph port makes the accumulation mechanism explicit in a way the original imperative loop only implied: `retrieve.py` emits `newly\_failed\_variants: Annotated[list[str], operator.add]` into `GraphState` — the exact reducer-typed field pattern Chapter 19B.4 introduced for concurrent fan-out — so that every parallel `Send` branch (one per query variant, Chapter 22C.8) that retrieves zero chunks contributes its own entry to the accumulated list, regardless of which other branches succeeded in the same run.

Figure 30.1 traces this record's full lifecycle end to end — not just the moment of failure, but everything that has to happen afterward for that single zero-chunk retrieval to actually change future behavior: accumulation during the run, persistence at the run's end, reloading at the start of a completely different process, and enforcement as a hard filter the next time the identical phrasing is attempted.

The failed-variant lifecycle, one session to the next

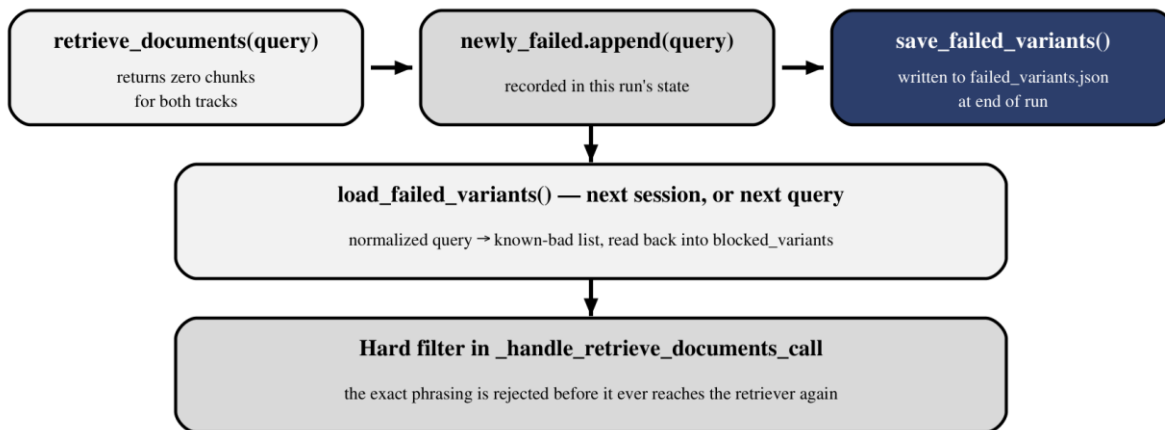


Figure 30.1 — A failure recorded during one run is written at session end and enforced as a hard filter at the very next retrieval attempt, in any future session.

30.4 Soft prompt injection vs. hard tool-layer filtering

The blocked-variant text injected into `_ROLE_AND_RULES` (Chapter 26.4) — "These exact phrasings retrieved 0 useful chunks the last time this question was asked. Do NOT reuse them" — is real and does measurably bias the model away from repeating a known-bad query. But Chapter 25.3's core lesson applies here without modification: a system prompt instruction is a bias on next-token probability, not a guarantee, and Chapter 24's dilution effects apply exactly as much to a cross-session blocklist injection as they did to the within-session `_all_tried_queries` case Chapter 27.4 covered.

The hard version is the identical pattern Chapter 27.4 already built for within-session dedup, now reading from `failed_variants.json` instead of an in-memory set: normalize the incoming query, check it against the loaded blocklist, and reject before the retriever is ever called, regardless of what the prompt said or whether the model's instruction-following held on this particular turn. Figure 30.2 makes the comparison explicit — the soft version can still fail under exactly the conditions Chapter 24-25 catalogued; the hard version cannot fail that way at all, because the model's cooperation was never a dependency in the first place.

Soft prompt injection vs. hard tool-layer filtering

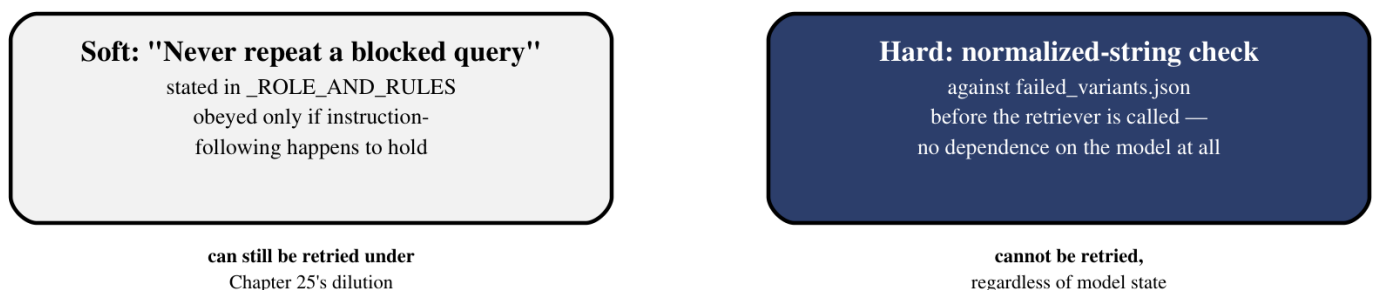


Figure 30.2 — The soft instruction biases the model; the hard filter does not depend on the model cooperating at all.

30.5 Walkthrough: load/save helpers wired into run_agent

The load side runs once, at the very start of a query, populating the `blocked` list that seeds `\_build\_system\_prompt()`'s soft injection and — in a fully hard-filtered design — a lookup set the retrieval handler checks directly. The save side runs once, at the very end of `run\_agent()`, folding `newly\_failed` (Chapter 27's local accumulator, populated by the same zero-chunk branch that already appends to `retrieved\_sources`) into the persisted file.

```
def load_failed_variants(path) -> dict:
    if not os.path.exists(path):
        return {}
    with open(path, encoding="utf-8") as f:
        return json.load(f)

def save_failed_variants(path, blocklist: dict, newly_failed: list[str]) -> None:
    now = datetime.now(timezone.utc).isoformat()
    for query in newly_failed:
        key = query.strip().lower()
        entry = blocklist.setdefault(key, {"count": 0, "last_seen": ""})
        entry["count"] += 1
        entry["last_seen"] = now
    with open(path, "w", encoding="utf-8") as f:
        json.dump(blocklist, f, indent=2)
```

Wiring this into `run\_agent()` is two lines: `blocked = load\_failed\_variants(path)` before the loop starts, seeding both the soft prompt injection and the hard-filter lookup set; `save\_failed\_variants(path, blocked, newly\_failed)` after the loop ends, persisting whatever this run discovered. The load-then-save-back pattern (rather than a pure append) is what lets `count` and `last\_seen` accumulate meaningfully across many sessions instead of each session only ever seeing its own additions.

30.6 Two real bugs in the port

This project's own migration from `app/agent\_query.py` to `app\_workflow/'s LangGraph implementation shipped exactly half of this chapter's loop at first. BUG-043 records it precisely: `user_input.py` read `failed_variants.json` on startup — the load side worked — but no node in the graph ever called the equivalent of `save_failed_variants()`. The blocklist worked *within* a session (variants were in state) but never grew *across* sessions, because the write half of the loop this section just walked through had simply never been ported.

Bug	Symptom	Root cause	Fix
BUG-043	Blocklist never grows across process restarts	Write-back node never ported from app/	New newly_failed_variants reducer field + save call in generate_answer.py
BUG-047	Failed variants written outside the project directory entirely	One extra ../ in a CWD-relative path constant	Rewrote path constants against a __file__-anchored project root

BUG-047 is a different failure mode entirely, but one this chapter's walkthrough makes easy to appreciate: `FAILED_VARIANTS_PATH` was built as a CWD-relative string with one extra `../` relative to its sibling path constants, so writes silently landed a directory above the actual project root. The blocklist file this section's `save_failed_variants()` writes to is only useful if every process, launched from every possible working directory, resolves to the *same* file — a lesson Chapter 27.6's single-source-of-truth principle applies just as much to a file path as to an in-memory counter.

What makes BUG-047 worth dwelling on is how quietly it fails. A missing write-back (BUG-043) produces an observable symptom fairly quickly — someone eventually notices the blocklist never grows. A path resolved one directory too high produces *no* error at all: `json.dump()` succeeds, the write completes, and the file that exists is simply not the file `load_failed_variants()` will look for next time. The bug is invisible until someone manually inspects the filesystem and finds the blocklist sitting one level above where every other `data/feedback/` file lives — exactly the class of failure `__file__`-anchored path resolution exists to make structurally impossible rather than merely rare.

Both bugs share a root cause worth naming explicitly: a feature with a read half and a write half, ported or refactored without verifying both halves still agree on where their shared file lives and whether both halves actually exist. `services.py` already used the safer `_project_root`-derived resolution pattern for `VECTOR_STORE_PATH` at the time BUG-047 was found — the fix was not inventing a new technique, it was applying a pattern that already existed elsewhere in the same codebase consistently, rather than letting one config module drift onto CWD-relative construction while its neighbors had already moved past it.

30.7 Verifying it works

Verifying this chapter's mechanism works end to end does not require anything exotic: run a known-zero-chunk query once, confirm `failed_variants.json` gained an entry, restart the process entirely, and ask the identical question again — the second run's debug log should show the hard filter rejecting the query before a single retrieval call is made, exactly the same `[DEDUP]`-style log line Chapter 27.4's within-session filter already produces, now firing on a query the *current* process never asked at all.

A more thorough verification checks both halves of the loop independently, precisely because BUG-043 demonstrated that one half can silently stop working while the other keeps functioning. Confirm the write path by inspecting the file directly after a zero-chunk run — `count` should have incremented for the normalized query, and `last_seen` should reflect the current run's timestamp. Confirm the read path separately by deleting or renaming the file, restarting, and observing that a previously-blocked query is now attempted again — proving that the blocking behavior genuinely depends on the file's contents rather than some other, coincidentally-correct code path.

This two-sided verification habit — checking that a persisted-state feature's write path actually wrote and its read path actually reads that same file, independently — is the general lesson this chapter's two bugs both teach. A feature that appears to work in a single continuous session can still be silently broken across the process boundary that matters most for a cross-session mechanism like this one, and the only way to catch that is to actually restart the process as part of the test, not merely trust that a feature working within one long-running session means it will still be there the next time the system starts cold.